Part A-1

Fibonacci

```
def fibonacci_iterative(n:int):
    if n == 1:
        return 0
    elif n == 2:
        return 1
    else:
        dp = [0] * n
        dp[0] = 0
        dp[1] = 1
        for i in range(2,n):
            dp[i] = dp[i-1] + dp[i-2]
        return dp[n-1]

def fibonacci_recursive(n):
    cache = {
        1:0,
        2:1
    }
    return helper(n,cache)

def helper(n:int,cache):
    if n in cache:
        return cache[n]
    else:
        return helper(n-1,cache) + helper(n-2,cache)


n = int(input("Enter value of n(nth Fibonacci number): "))
print(f"Fibonacci Number(Iterative): {fibonacci_iterative(n)}")
print(f"Fibonacci Number(Recursive): {fibonacci_recursive(n)}")
```

Output

Fibonacci sequence:

0, 1, 1 ,2 ,3, 5, 8.

Time and Space Complexity

- The time complexity of the above code is **T(2^N) i.e, exponential**.

Part A-2

Output

```
AAAAAAABCCCCCCDDEEEEE
symbols:  dict_keys(['A', 'B', 'C', 'D', 'E'])
probabilities:  dict_values([7, 1, 6, 2, 5])
symbols with codes {'A': '00', 'C': '01', 'E': '10', 'D': '110', 'B': '111'}
Space usage before compression (in bits): 168
Space usage after compression (in bits): 45
Encoded output 00000000000000111010101010111011010101010
```

Part A-3

Code:- class Item:

def init (self, value, weight): self.value = value self.weight = weight

def fractionalKnapsack(W, arr):

Sorting Item on basis of ratio

arr.sort(key=lambda x: (x.value/x.weight), reverse=True)

Result(value in Knapsack) finalvalue = 0.0

Looping through all Items for item in arr:

If adding Item won't overflow, # add it completely

if item.weight <= W:

W -= item.weight finalvalue += item.value

If we can't add current Item, # add fractional part of it else:

finalvalue += item.value * W / item.weight break

Returning final value return finalvalue

Driver Code

if name == " main ":

W = 50

arr = [Item(60, 10), Item(100, 20), Item(120, 30)]

Function call

max_val = fractionalKnapsack(W, arr) print(max_val)

Output

Maximum value we can obtain = 24

Part A-4

```
# A Dynamic Programming based Python program
# for the 0/1 Knapsack Problem
def knapSack(W, wt, val, n):
    # Making the DP array
    dp = [0 for i in range(W + 1)]
    # Taking the first i elements
    for i in range(1, n + 1):
        # Starting from the back
        for w in range(W, 0, -1):
            # Check if the item can be included
            if wt[i - 1] <= w:
                # Finding the maximum value
                dp[w] = max(
                    dp[w],
                    dp[w - wt[i - 1]] + val[i - 1] )
    # Returning the maximum value
    return dp[W]
# Driver code
val = [60, 100, 120] wt = [10, 20, 30]
W = 50 n = len(val) print(knapSack(W, wt, val, n))
```

Output
220

Part A:5 N-Queens

```
N = 4 def solve(board, col):
    if col == N:
        return True
    for row in range(N):
        if all(board[row][c] == 0 and
               row - c != r - col and
               row + c != r + col
               for r in range(N) for c in range(col)
               if board[r][c] == 1):
            board[row][col] = 1
            if solve(board, col + 1):
                return True
            board[row][col] = 0
    return False
board = [[0] * N for _ in range(N)]
solve(board, 0) for row in board  print(*row)
```

Output

```
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
```